

Paper number: P1704r0

Date: 2019-06-16

Author: Joshua Berne, Andrzej Krzemieński

Email: akrzemi1 (at) gmail (dot) com

Audience: EWG, CWG

Undefined functions in axiom-level contract statements

Current [\[WD\]](#) allows axiom-level contract statements to be evaluated at runtime. This is contradictory with the design goals for axiom-level contract statements outlined in [\[P0380r0\]](#), which explicitly notes that the goal for axiom-level contract statements is to allow functions without definitions. In this paper we propose to make it clear that a program is guaranteed, in any build level, to compile and link when conditions in axiom-level contract statements contain references to objects and functions without definitions.

Non-goals

At the same time we want to preserve the guarantee from [\[WD\]](#) that if the implementation can somehow determine the value that the predicate would return, it should be able to use this information for optimization or correctness verification purposes. Unconditionally having such optimizations are controversial and considered a deficiency in [\[WD\]](#) by some parties, but we consider it an orthogonal issue to the one discussed in this paper.

In this paper we do not discuss side effects being allowed in axiom-level contract statements. That will be the subject of another paper.

What we need

We need to be able to declare predicates like:

```
template <InputIterator I, Sentinel<I> S>
    bool is_reachable(I b, S e); // never defined

template <typename T>
    bool points_to_heap_object(T* p); // never defined
```

They are unimplementable, therefore they are only declared but never defined. They are still useful for static analysis. We want to be able to put them in axiom-level contract statements and have the guarantee that we will never get a linker error saying that an odr-used symbol is missing, regardless of any build level.

At the same time, it is plausible that the implementation can understand the semantics of our condition and by some other means it can compute the result without causing any side effects and without odr-using any new entity in the condition. Such result could be used for optimization purposes, or for additional correctness validation. The current [\[WD\]](#) allows this and we do not want to prevent this.

Note that with this change contract statements with level `axiom` will become substantially different from other levels: only for them the “no linkage problems” guarantee applies. In contrast, `audit` and `default` levels require definitions even if they are not evaluated in the current build level. For those levels we explicitly prefer retaining that ODR use to minimize the risk of writing code that links with some build levels and fails to link with others, which is a significant benefit contracts can have as a language facility over being implemented solely with macros.

To summarize our goal:

1. Missing definitions of entities (objects or functions) in the condition of an axiom-level contract statement never make the program ill-formed. Same as inside `sizeof()` or `decltype()`.
2. Optimizations still potentially enabled if there is a way to determine the predicate result without violating point 1.

Proposed wording

Modifications apply to section `[dcl.attr.contract.cond]`.

Modify paragraph 4:

During constant expression evaluation, only predicates of checked contracts are evaluated. In other contexts, it is unspecified whether the predicate for a `default` or `audit` level contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to `false`, the behavior is undefined.

Add paragraph at the end:

The predicate `p` of a contract condition with `axiom` *contract-level* is an unevaluated operand. If an implementation is able to determine what the value would be returned by evaluating `p`, and this value is `false`, the behavior is undefined.

[Example:

```
bool pred(int i); // never defined

void f(int * p)
    [[expects axiom: p && pred(*p)]];

void g(int * p)
{
```

```
f(p);  
if (p != nullptr) // the check can be elided  
    *p = 0;  
}
```

Implementation can eliminate the check `p != nullptr` in function `g`. If `p` was null, the precondition of `f` would evaluate to `false`, which would be undefined behavior. The potential contract violation can be determined without the call to function `pred` owing to the semantics of operator `&&`. – *end example*]

Acknowledgements

Many people in EWG reflector helped shape this proposal, in particular, Tony Van Eerd.

References

[WD] – Richard Smith, N4800, “Working Draft, Standard for Programming Language C++”.

[P0380r0] – G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, “A Contract Design”.

[P1429r1] – Joshua Berne, John Lakos, “Contracts That Work”.